Système et Réseaux : TP5

février 2026 — Pierre Rousselin et Antoine Rozenknop

# 1 Problèmes à savoir traiter

**Exercice 1 : Belmondo, de Funès et *mutex***

1. Télécharger et lire le programme `belmondo.c` sur l'ENT.
2. L'exécuter plusieurs fois : que constatez-vous ?
3. Modifier le programme de façon à ce que toutes les lignes du texte initial soient lues et que les acteurs ne parlent pas en même temps. Plutôt que d'obtenir un affichage de ce genre :

```
jJEj eTeE   TtIteENe S
ttiieennssT
U ME TIENS
ppaarrP A Rl lLaAa  B AbRbBaIaCrHrEbTbTiEi
cchheetDtEt NtOeUSe
DEUX
qquuiQiU I  rRrIiRiA
rraaT
A
ppeetPtEtTtTeEe
```

On veut quelque chose comme :

```
JE TE TIENS
ttuu  mmee  ttiieennss
PAR LA BARBICHETTE
llee  pprreemmiieerr
DE NOUS DEUX
qquuii  rriirraa
AURA UNE
ttaa
PETTE
```

Peu importe si un des acteurs enchaîne deux lignes avant que l'autre n'ait fini de traiter la sienne, ou si les lignes ne sont pas dans le bon ordre (on rectifiera au montage), l'essentiel est qu'elles y soient toutes.

--- * ---

**Exercice 2 : Barrière (non réutilisable)**

1. Télécharger sur l'ENT, lire, compiler et exécuter (plusieurs fois) le programme `barriere_non-sync.c`.
2. On voudrait que tous les chiffres soient affichés avant toutes les lettres (mais pas nécessairement dans l'ordre). Par exemple, ceci est acceptable pour $N = 5$ :

```
3
4
2
1
0
A
E
B
D
C
```

En utilisant variable(s) supplémentaire(s) et/ou sémaphores et/ou mutex, modifier le programme de façon à satisfaire la condition ci-dessus.

Indication : lorsque plusieurs personnes se suivent et passent une porte, il arrive que chacun tienne la porte pour le suivant...

--- * ---

**Exercice 3 : Double-buffering**

Le programme `space_invaders.c`, disponible sur moodle fait les choses suivantes `num_iter` fois :

— D'abord la prochaine image à afficher est calculée. Ici, simplement, toutes les lignes sont décalées et un pixel (représenté par `*`) est ajouté sur la première ligne.

— Ensuite, le terminal est effacé et l'image (les pixels de l'écran) est affichée.

1. Télécharger, lire, compiler et exécuter ce programme.
2. Votre mission maintenant est de rendre ce programme concurrent : un *thread* devrait être dédié au calcul du prochaine écran (on dit aussi de la prochaine *frame*) pendant qu'un autre affiche la *frame* courante.

   Vous aurez pour cela besoin de 2 écrans, un pour la `frame` en train d'être affichée (il sera lu par les deux *thread*) et un autre pour la prochaine (il sera modifié par un seul *thread*), en train d'être calculée. Pour éviter les copies intempestives, vous travaillerez également avec deux pointeurs globaux :

   ```
   int (*ecran_a_afficher)[HAUTEUR][LARGEUR];
   int (*ecran_a_modifier)[HAUTEUR][LARGEUR];
   ```

   chacun contenant l'adresse d'un des deux écrans. Un des *threads* se chargera d'échanger les valeurs des deux pointeurs au bon moment.
3. Il faudra bien sûr faire bien attention aux problèmes de synchronisation. Lancer le programme avec double-buffer avec la commande `time` devrait montrer qu'il prend (environ) deux fois moins de temps que la version sans double-buffer.

Pour plus d'informations sur cette technique répandue appelée *double-buffering*, voir `https://en.wikipedia.org/wiki/Multiple_buffering`

--- * ---

## 2 Problèmes plus difficiles

**Exercice 4 : Le problème du rendez-vous (à N), ou barrière réutilisable**

On reprend l'exercice du TP précédent, mais cette fois avec $N > 1$ *threads*.

1. Télécharger, lire, compiler et exécuter le fichier `rendez_vous_a_N_non-sync.c` sur la page du cours. Constater que les *threads* ne se donnent vraiment pas rendez-vous.
2. On veut, comme dans l'exercice précédent, que les *threads* se donnent rendez-vous pour que, pour chaque $k$, les affichages de l'entier $k$ soient groupés. Par exemple avec $N = 3$, autoriser

   ```
   a : 0
   c : 0
   b : 0
   b : 1
   ```

   mais interdire

   ```
   a : 0
   b : 0
   a : 1
   c : 0
   ```

   Indications :

   — On peut utiliser une variable globale (à manipuler correctement) pour compter, à chaque tour, le nombre de *threads* qui ont terminé de prendre leur temps pour compter puis dans un deuxième temps de compter le nombre de *threads* qui ont terminé d'afficher.

   — Le dernier a avoir fini chaque étape a donc un rôle à jouer.

   — Avec une structure, on pourra transmettre à chaque *thread* les adresses des objets utilisés pour la synchronisation et leurs indices respectifs.

- On utilisera entre autres deux sémaphores : un qui permet d'attendre que tout le monde ait fini de prendre son temps pour compter avant d'afficher et un autre qui permet d'attendre que tout le monde ait fini d'afficher avant de passer à l'itération suivante.
- Penser à deux portillons : un pour entrer et un pour sortir.

--- * ---

**Exercice 5 : Le problème des danseurs (difficile)**

Dans ce problème, il y a des danseurs, dont `NB_MENEURS` meneurs et `NB_SUIVEURS` suiveurs. Un meneur ne peut danser qu'avec un suiveur et réciproquement (c'est une danse de salon, par exemple du *rock'n roll*). Dans une boucle infinie :

- Chaque danseur met un certain temps à se préparer.
- Ensuite, il essaie de choisir un partenaire (un suiveur, s'il est meneur et réciproquement) :
  - si aucun partenaire n'est disponible, il ne pourra en choisir un : il attend d'être choisi en s'adossant tristement au mur ;
  - sinon, il choisit (c'est-à-dire ici réveille : on laisse faire le système) un partenaire de l'autre catégorie et l'invite à danser, après lui avoir demandé son nom (ici son numéro) et donné le sien.
- Lorsqu'ils ont choisi ou ont été choisis les danseurs dansent avec leur partenaire.

**1.** Télécharger et lire le programme `danseurs_non_sync.c` sur moodle.

**2.** Le compiler et l'exécuter. Noter un certain nombre d'incohérences.

**3.** Le but est d'arriver à la situation décrite ci-dessus : un danseur choisit ou est choisi (réveille ou est réveillé) par un danseur de l'autre catégorie et danse avec lui.

C'est loin d'être facile, alors voici quelques pistes :

a) les nombres des meneurs et des suiveurs en attente devraient être enregistrés, par exemple dans des variables globales ;

b) à tout instant, il ne peut y avoir qu'un et un seul danseur qui essaie d'en choisir (réveiller) un autre et manipule l'une ou l'autre des deux variables évoquées ci-dessus ;

c) on peut utiliser un sémaphore `file_suiveur` pour les suiveurs qui attendent d'être choisis (réveillés) par un meneur et un sémaphore `file_meneur` pour les meneurs qui attendent d'être choisis (réveillés) par un suiveur ;

d) il faut également faire attention à ce que les partenaires se transmettent correctement leurs noms (numéros).

--- * ---